

Roteiro Completo de Apresentação - Compilador GYH (Parte do Felipe)

✂️ Preparação Antes de Gravar

1. Deixe abertos no seu editor: `GyhGrammar.g4`, `SymbolTable.java`, `Symbol.java`.
 2. Deixe preparados os testes: `programa_lexico.gyh`, `programa6.gyh` e `programa2.gyh`.
 3. Lembre-se de apontar o mouse para as linhas de código quando estiver falando sobre elas.
-

🎬 Início da Gravação

[Ação] Abra a câmera/microfone e mostre a IDE com o arquivo `GyhGrammar.g4` aberto na **linha 209** (onde começam os tokens).

Felipe (Fala Exata): "Olá, professora. Eu sou o Felipe Rodrigues e juntamente com o Guilherme, nós desenvolvemos o nosso Compilador da linguagem GYH. Eu fiquei responsável por toda a parte da análise Léxica, análise Sintática e também pela análise Semântica do projeto, e é isso que eu vou demonstrar agora."

1. Análise Léxica (Explicando o Código)

[Ação] Aponte o mouse para os tokens a partir da **linha 209** no `GyhGrammar.g4` até a **linha 242**.

Felipe (Fala Exata): "Começando pelo Analisador Léxico, nós utilizamos o próprio ANTLR para mapear as expressões regulares da linguagem. Aqui no final do arquivo `GyhGrammar.g4`, a partir da linha 209, nós declaramos todos os tokens lexicais. Por exemplo, temos as palavras reservadas como `PCInt`, `PCReal`, e `PCLer`. Na linha 237 nós definimos o token `Var` que aceita letras seguidas de letras ou números, formando nossos identificadores. Logo abaixo temos `NumInt` e `NumReal`. Um detalhe muito importante está nas linhas 241 e 242: nós utilizamos o comando `-> skip` nos comentários e nos espaços em branco (`WS`). Isso significa que o analisador léxico descarta os espaços e comentários automaticamente, entregando para o analisador sintático uma cadeia limpa de tokens válidos."

2. Análise Sintática (Explicando o Código)

[Ação] Suba o arquivo `GyhGrammar.g4` até a **linha 64** (onde começa a regra `programa`).

Felipe (Fala Exata): "Subindo aqui para a linha 64, entramos no Analisador Sintático. Nós estruturamos a gramática exatamente conforme a especificação. A regra principal `programa` exige que o código tenha a estrutura estrita de um delimitador, a palavra `DEC`, depois a nossa regra de `listaDeclaracoes`, outro delimitador, e a palavra `PROG`, seguida da `listaComandos`. Tanto a `listaDeclaracoes` na linha 73 quanto as nossas expressões aritméticas utilizam recursividade para aceitar múltiplos comandos. Um ponto forte de usar o ANTLR v4 é que ele já trata a recursividade à esquerda nativamente, então nossas regras de

precedência nas expressões matemáticas funcionam sem precisarmos criar regras intermediárias complexas."

3. Análise Semântica (Explicando o Código)

[Ação] Troque a aba para o arquivo `SymbolTable.java` e depois `Symbol.java`. Em seguida, volte para o `GyhGrammar.g4` na **linha 16** (bloco `@members`).

Felipe (Fala Exata): "Já para o Analisador Semântico, eu implementei as validações acopladas diretamente na gramática usando injeção de código Java puro. Eu criei as classes `Symbol` para armazenar o nome e o tipo da variável, e a classe `SymbolTable`, que utiliza um `HashMap` em Java para manter a nossa tabela de símbolos na memória. Voltando aqui pro `GyhGrammar.g4`, na linha 16, no bloco `@members`, nós instanciamos essa `SymbolTable`. Aqui eu desenvolvi os métodos semânticos principais: na linha 29, o `verificaEDeclaraVariavel` garante que a variável seja inserida na tabela e alerta se houver redeclaração. E na linha 39, o método `verificaUsoVariavel` consulta o `HashMap` pra verificar se a variável que o usuário está tentando usar em uma operação ou atribuição realmente existe. Nós chamamos essas funções injetadas direto nas regras sintáticas, como na regra `declaracao` da linha 76."

4. Demonstração dos 3 Erros no Terminal

[Ação] Abra o terminal da sua IDE para começar a rodar os testes. Abra a aba com o arquivo `programa_lexico.gyh`.

Felipe (Fala Exata): "Agora eu vou provar o funcionamento dessas 3 análises na prática mostrando a captura dos erros. Primeiro, o Erro Léxico: neste arquivo `programa_lexico.gyh`, eu coloquei um caractere de arroba (@) perdido na atribuição. Vou rodar o compilador..." [Ação] Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "Como podem ver no console, o lexer captura imediatamente e acusa o token `recognition error at: '@'`, rejeitando o caractere inválido."

[Ação] Troque para a aba do `programa6.gyh` (que tem o erro do igual duplo). **Felipe (Fala Exata):** "Agora o Erro Sintático: no `programa6.gyh`, na linha 10, tentaram fazer uma atribuição de variável usando o operador relacional `==` ao invés de `:=`. Vou rodar o compilador..." [Ação] Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "O analisador sintático falha exatamente na linha 10, gerando o erro *no viable alternative at input 'fatorial =='*. Ele percebeu que a sequência de tokens violou a regra de atribuição."

[Ação] Troque para a aba do `programa2.gyh`. Aponte para a linha 15 (onde a variável `num4` é usada). **Felipe (Fala Exata):** "E por último, o Erro Semântico: no `programa2.gyh`, dentro do condicional na linha 15, o código está tentando usar a variável `num4`. Porém, ela não foi declarada no bloco `DEC`. Vou rodar..." [Ação] Rode o comando para compilar o arquivo. **Felipe (Fala Exata):** "O console nos mostra perfeitamente a mensagem da nossa classe Java: *Erro Semântico: Variável não declarada! -> num4*. A nossa tabela de símbolos impediu a geração do código sujo."

[Ação] Conclusão da sua parte. **Felipe (Fala Exata):** "Como demonstrei, as 3 fases de análise, Léxica, Sintática e Semântica estão funcionando rigorosamente. Agora eu passo a palavra para o Guilherme, que vai apresentar como nós implementamos a arquitetura de Geração de Código utilizando essas regras, e vai rodar a compilação final dos programas válidos para C. Muito obrigado!"

(Fim da sua parte no vídeo!)